

Parallel Computation of Generalized Hypertree Decompositions

Georg Gottlob^{1,2} Cem Okulmus¹ Reinhard Pichler¹

¹Institute of Logic and Computation,
TU Wien, Austria

²Department of Computer Science
University of Oxford, UK

13th Alberto Mendelzon International Workshop
on Foundations of Data Management
June 05, 2019

Introduction

Decomposition Algorithm

Parallelization

Results

Next Steps

- A quick overview over the setting and motivation

Three Problems

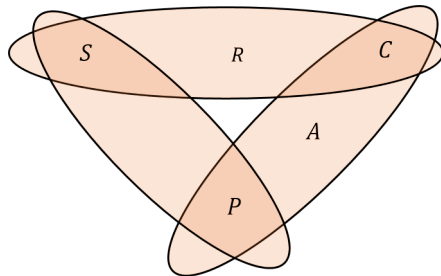
- Conjunctive Query Evaluation
- Constraint Satisfaction Problem
- Homomorphism Problem

All these problems are essentially the same

All these problems are based on hypergraphs.

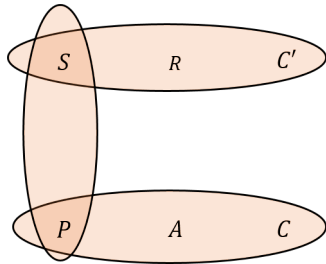
Query

Is there any teacher having a child enrolled in her course?

$$ans \leftarrow Enrolled(S, C, R) \wedge Teaches(P, C, A) \wedge Parent(P, S)$$


Query

Is there any teacher having a child enrolled in **some** course ?

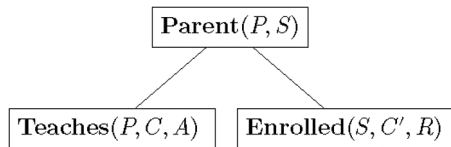
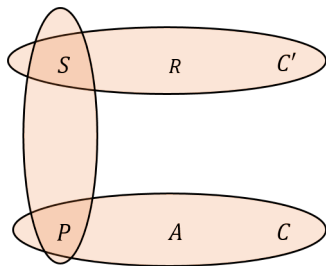
$$ans \leftarrow Enrolled(S, C', R) \wedge Teaches(P, C, A) \wedge Parent(P, S)$$


Acyclic CQs (cont.)

Query

Is there any teacher having a child enrolled in **some** course ?

$$ans \leftarrow Enrolled(S, C', R) \wedge Teaches(P, C, A) \wedge Parent(P, S)$$



Join Tree

- **NP-complete in the general case**
(Chandra and Merlin '77)
- **Tractable in case of acyclic CQs**
(Yannakakis '81)

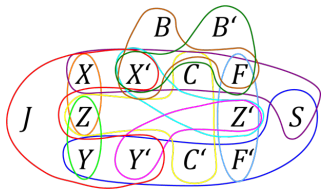
Interest in larger tractable classes in CQs

Tree Decomposition

$$ans \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$$

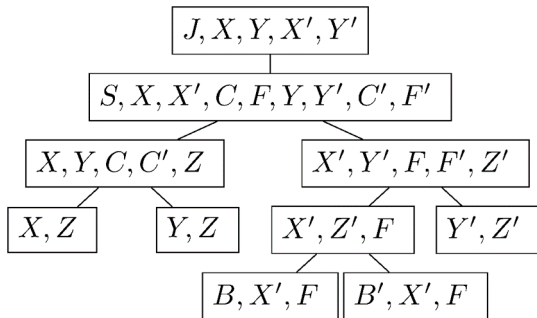
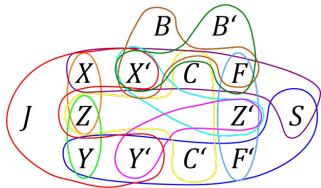
Tree Decomposition (cont.)

$ans \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$



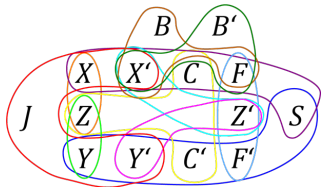
Tree Decomposition (cont.)

$ans \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$

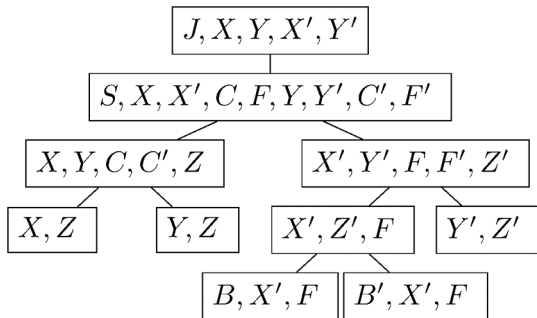


Tree Decomposition (cont.)

$ans \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$

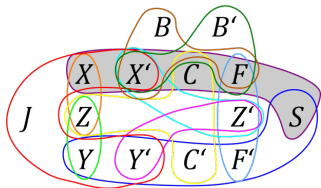


- Variables of each atom covered by some node

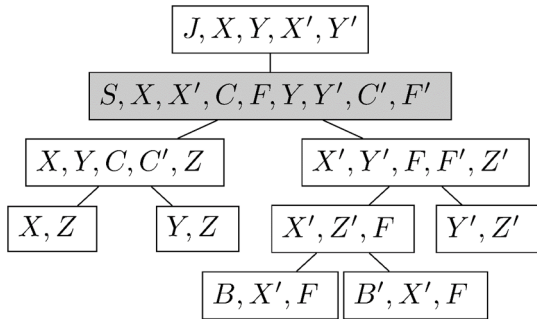


Tree Decomposition (cont.)

$ans \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$

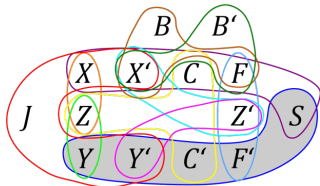


- Variables of each atom covered by some node

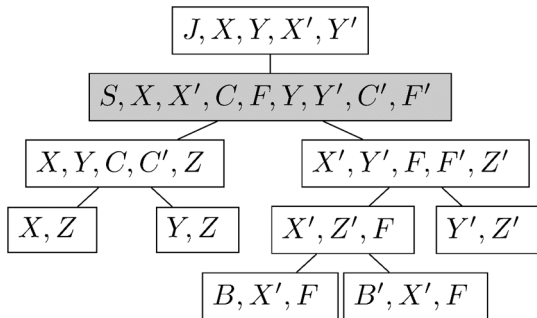


Tree Decomposition (cont.)

$ans \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$

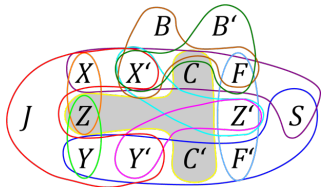


- Variables of each atom covered by some node

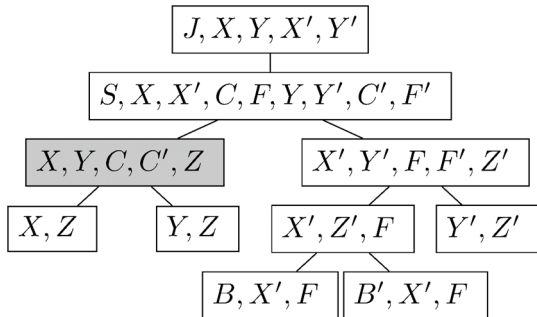


Tree Decomposition (cont.)

$ans \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$

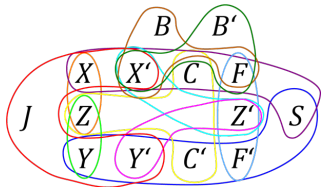


- Variables of each atom covered by some node

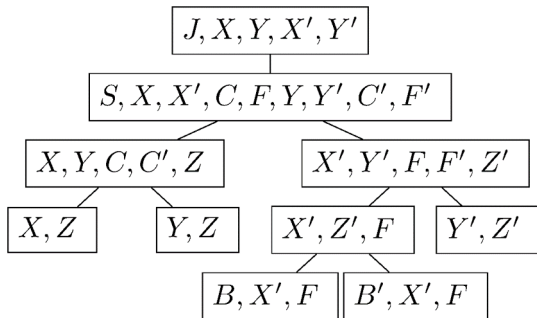


Tree Decomposition (cont.)

$ans \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$

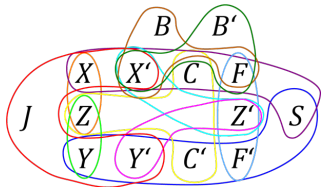


- Variables of each atom covered by some node
- Connectedness Condition

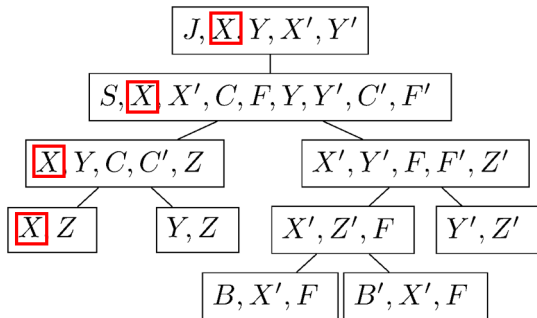


Tree Decomposition (cont.)

$ans \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$

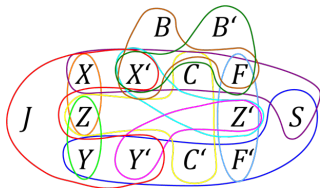


- Variables of each atom covered by some node
- Connectedness Condition

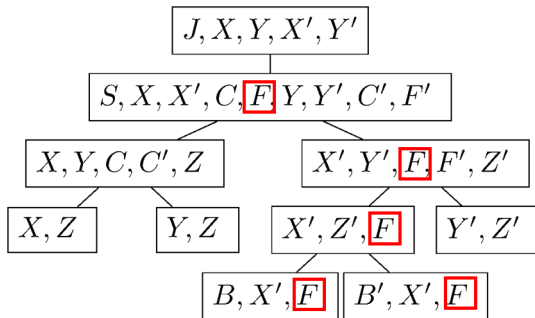


Tree Decomposition (cont.)

$ans \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$

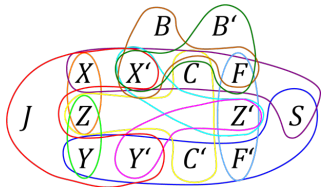


- Variables of each atom covered by some node
- Connectedness Condition

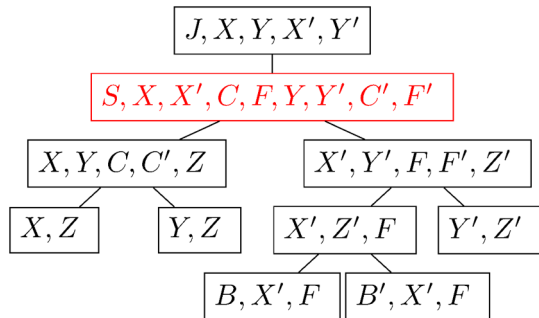


Tree Decomposition (cont.)

$ans \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$



- Variables of each atom covered by some node
- Connectedness Condition



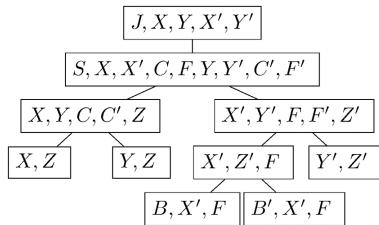
Tree Width: 8

- Consider a set of vertices $V' \subseteq V(H)$
- An edge cover is a set of edges $E' \subseteq E(H)$, s.t.
all vertices in V' are “covered” by E' , i.e. $V' \subseteq \bigcup_{e \in E'} e$
- Add edge covers to the tree decomposition

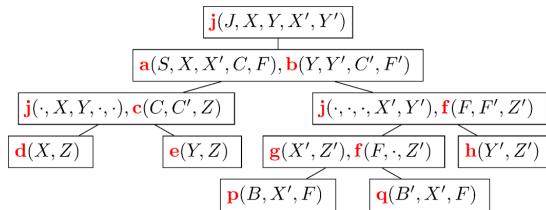
Generalized Hypertree Decomposition (GHD)

Tree Decomposition + Edge Covers

$ans \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$



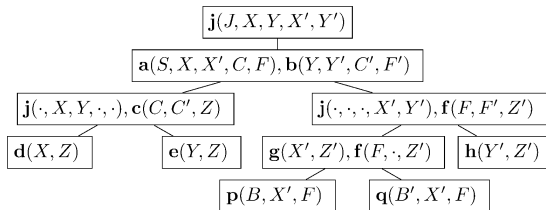
Tree Width: 8



Generalized Hypertree Width (ghw): 2

Hypertree Decompositions

GHD + Special Condition

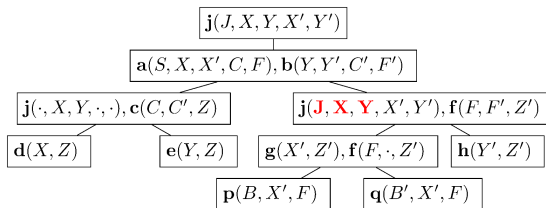


Generalized Hypertree Width (hw): 2

Hypertree Decompositions

GHD + Special Condition

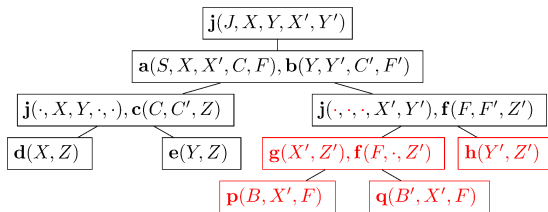
- Each variable that **disappeared** at some node **n**



Hypertree Decompositions

GHD + Special Condition

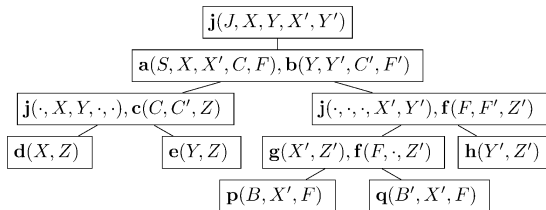
- Each variable that **disappeared** at some node **n**
- does **not reappear** in the subtree rooted at **n**



Hypertree Decompositions

GHD + Special Condition

- Each variable that **disappeared** at some node **n**
- does **not reappear** in the subtree rooted at **n**

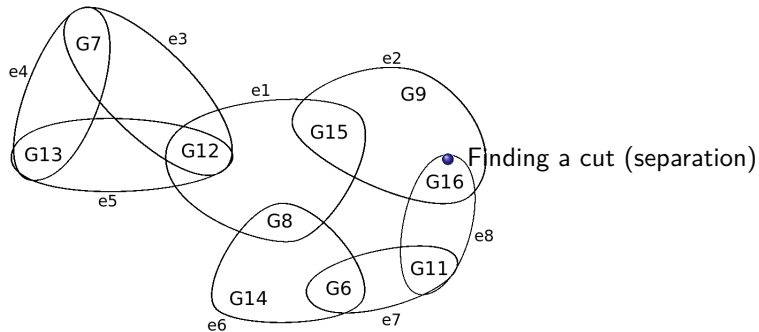


Hypertree Width (hw): 2

Separator

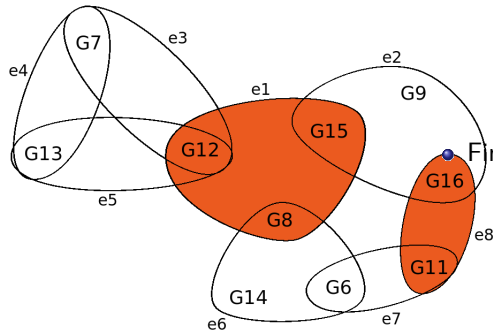
- We can think of a set of edges $S \subseteq E(H)$ as a **separator**
- Idea: Consider $E(H) \setminus S$ and see which edges still connected
- Will get one (or more) connected components C_i
- The **size** of C_i is $|\{e \mid e \in C_i\}|$

Decomposition Algorithm (1/5)



Decomposition Algorithm (2/5)

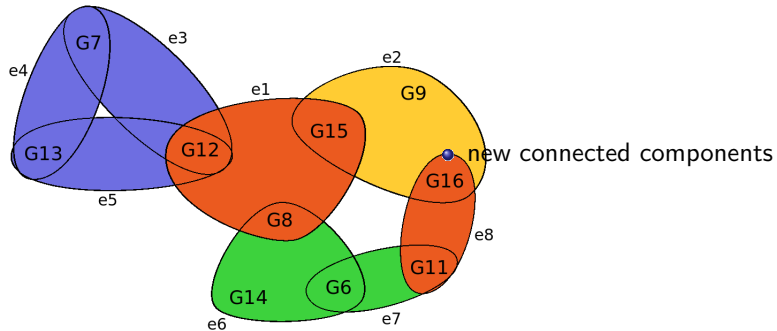
e1, e8



Finding a cut (separation)

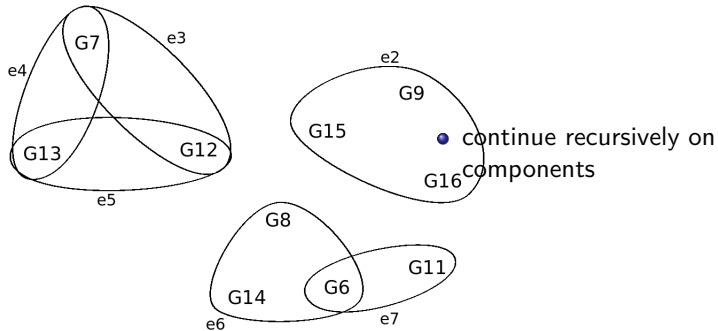
Decomposition Algorithm (3/5)

e1, e8

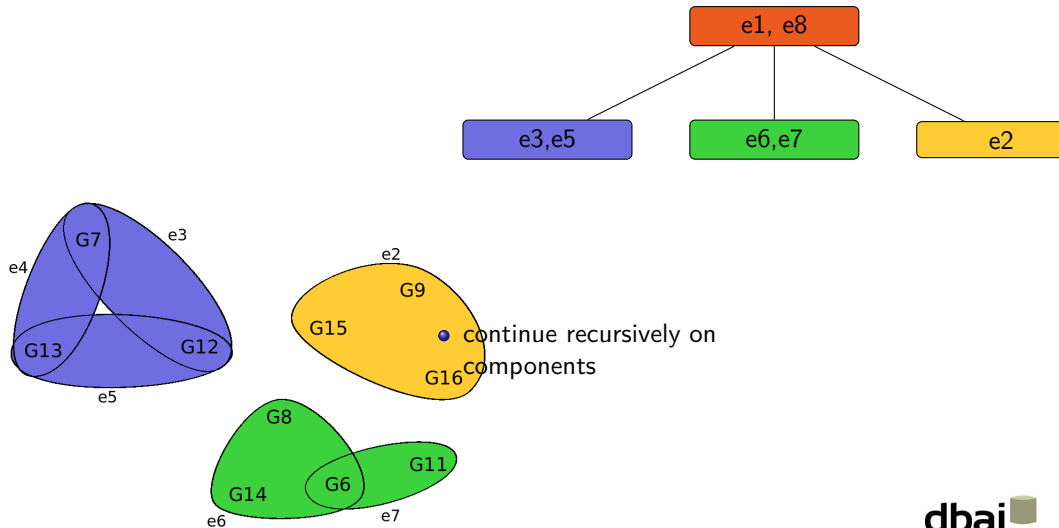


Decomposition Algorithm (4/5)

e1, e8



Decomposition Algorithm (5/5)



Balanced Separator

Balanced Separator

A separator S s.t. all for all its connected components C_i $1 \leq i \leq n$ we have $|C_i| \leq \frac{|E(H)|}{2}$ is called a **balanced separator**

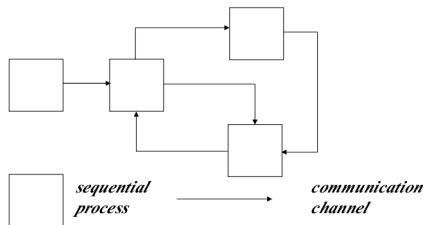
Theorem (Adler et al. '07)

A hypergraph H has **no** balanced separator of size $\leq k$
 $\Rightarrow H$ has no GHD of width $\leq k$

- Used by Fischl et al. (PODS 2019) to compute Generalized Hypertree Decompositions

Parallelization

- All work on computing GHDs so far sequential
- Many instances (with high width) not solved exactly
- **Goal**
Use Balanced Separator algorithm for a parallel implementation
- Implemented in Go (designed by Google in 2009)
 - concurrent programming scheme (goroutines)



- Finding a **good design** for parallelizing the algorithm
 - The parallel search for balanced separators
 - The recursive calls on resulting components
- Partitioning the work space equally
- Efficiently support backtracking
- Splitting resources equally during recursive call

Parallel Search for Separators

- Using **two types of goroutines**
- **“Central” goroutine**
 - oversees the search
 - creates workers
 - waits on two conditions: 1) the workers find a balanced separator or 2) all the workers stop → nothing was found
- **Worker goroutine**
 - Gets a fixed share of work (produced by central goroutine)
 - Goes over each item in its work pool
 - sends the result to central if it is a balanced separator
- Workers save their current workpool, continue when backtracking

Parallelizing Recursive Calls

- First idea:
Simply spawn each recursive call in its own goroutine.
- Problem:
Does not consider how resources (for separator search) are shared *among* recursive calls
- **Goal**
Develop and implement a way s.t. all recursive calls are worked on to the same degree (would result in speedup during negative instances, as reject found faster)
- Challenge:
Finding a design that works with all the prior challenges as well

Results

- Our implementation already solved some new instances
 - **Goal** “fill in the blank spots” remaining from HyperBench
- For comparing speedup, we considered sequential version of the same algorithm
 - Tests ran on machine with 12-core CPU

- Our implementation already solved some new instances
 - **Goal** “fill in the blank spots” remaining from HyperBench
- For comparing speedup, we considered sequential version of the same algorithm
 - Tests ran on machine with 12-core CPU

graph	sequential	parallel	speedup
Dubois-016.xml.hg	668.097 ms	45.39 ms	14.71
rand-25-10-25-87-24.xml.hg	5936.83 ms	879.99 ms	7.84
Nonogram-012-table.xml.hg	73976.08 ms	11057.68 ms	6.69
Pi-20-10-20-30-17.xml.hg	1439.71 ms	200.54 ms	7.17

Running times of b-seps algorithm for width 3.

Next Steps

- Looking at running recursive calls on separate machines
 - Exploring the move to a “share-nothing” architecture
- Preprocessing the hypergraph
 - GYÖ reduct, Type collapse
- Exploring new heuristics
 - Ordering edges by some measure to quickly find good separators
- Combining BalSep algorithm with DetKDecomp
 - DetKDecomp showed better results for positive instances, combined approach effective for negative *and* positive instances ?
- Perfect Separators
 - Find a separator s.t. no need to backtrack in case of reject